

Interface Prototype（確定版）

確定日: 2026-05-23

ベースドキュメント: [myIRCd/docs/interface.md](#)

ステータス: ✔ 確定（MTG用）

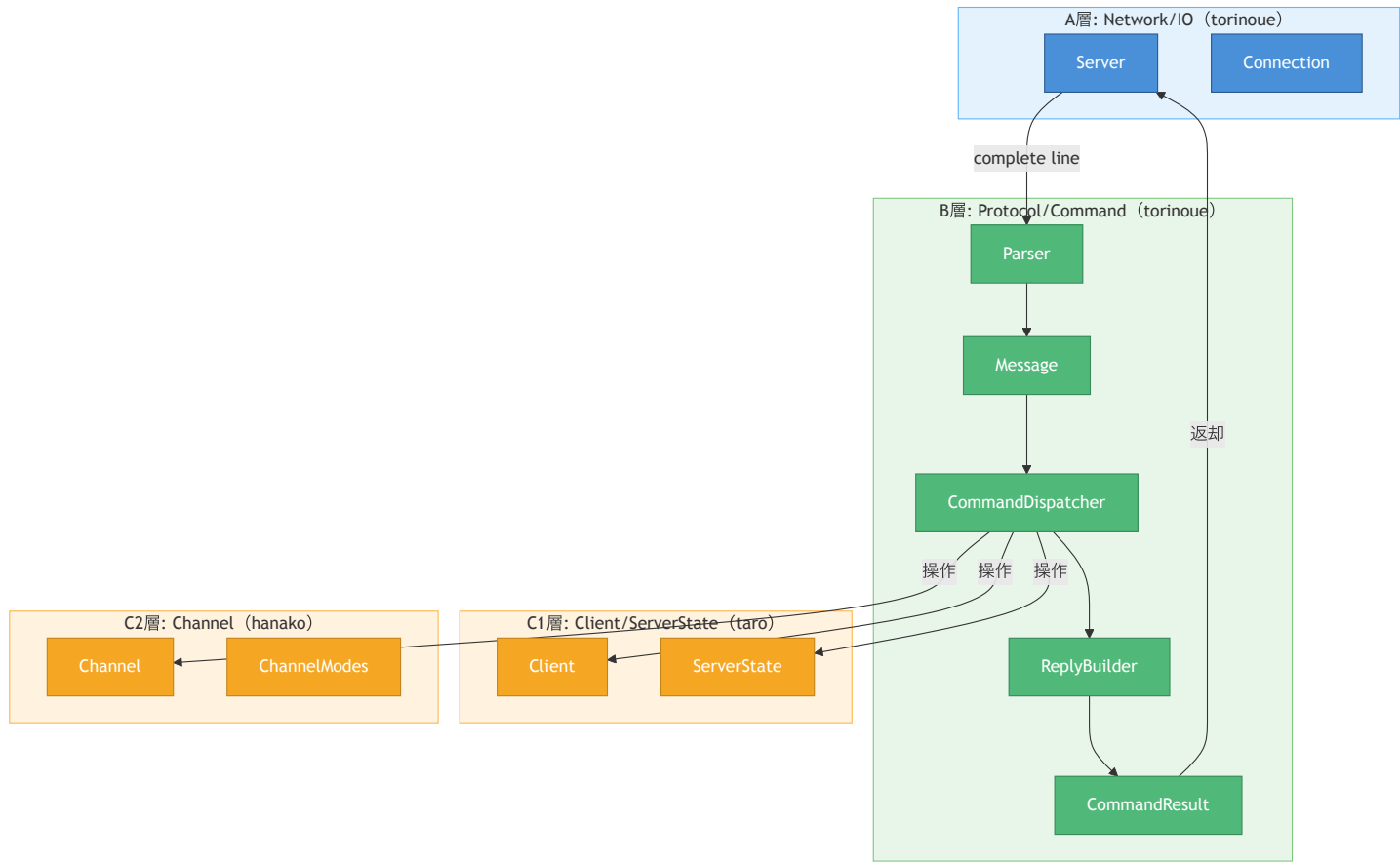
チーム構成

担当	メンバー	範囲
A	torinoue	Network / IO（Server, Connection）
B	torinoue	Protocol / Command（Parser, Dispatcher, ReplyBuilder）
C1	taro	Client / ServerState
C2	hanako	Channel / ChannelModes

1. 本ドキュメントの位置づけ

- ベース: [myIRCd/docs/interface.md](#) の全内容を継承
- 目的: taro/hanako向けのインターフェース確定・合意
- 変更管理: 本ファイルで追記・修正を管理。元ファイルは変更しない

2. 担当間依存関係（図解）



3. C1層インターフェース（taro担当）

詳細は [interface.md Section 7](#) 参照

3.1 Client

関数	戻り値	呼び出し元	説明
<code>int fd() const</code>	<code>int</code>	B, C2	fdを取得
<code>const std::string& nick() const</code>	<code>const std::string&</code>	B, C2	nickを取得
<code>const std::string& username() const</code>	<code>const std::string&</code>	B	usernameを取得
<code>const std::string& realname() const</code>	<code>const std::string&</code>	B	realnameを取得
<code>void setUsername(const std::string&)</code>	<code>void</code>	B	username設定
<code>void setRealname(const std::string&)</code>	<code>void</code>	B	realname設定
<code>void setPassOk(bool)</code>	<code>void</code>	B	PASS成功状態設定
<code>bool isPassOk() const</code>	<code>bool</code>	B	PASS済みか
<code>bool isRegistered() const</code>	<code>bool</code>	B	登録完了か
<code>bool canRegister() const</code>	<code>bool</code>	B	登録可能か（PASS/NICK/USER揃った）
<code>void markRegistered()</code>	<code>void</code>	B	登録完了にする

⚠ 注意: `setNick()` は外部から直接呼ばない。`ServerState::updateNick()` を使う。

3.2 ServerState

関数	戻り値	呼び出し元	説明
<code>const std::string& password() const</code>	<code>const std::string&</code>	B	サーバパスワード
<code>void addClient(int fd)</code>	<code>void</code>	Server	Client作成
<code>void removeClient(int fd)</code>	<code>void</code>	Server, B	Client削除（Channel参照も除去）
<code>Client* getClientByFd(int fd)</code>	<code>Client*</code>	B	fdからClient取得
<code>Client* getClientByNick(const std::string&)</code>	<code>Client*</code>	B	nickからClient取得
<code>bool nickExists(const std::string&) const</code>	<code>bool</code>	B	nick重複確認
<code>void updateNick(Client&, const std::string&)</code>	<code>void</code>	B	nick変更（辞書も更新）
<code>Channel* getChannel(const std::string&)</code>	<code>Channel*</code>	B	Channel取得（なければNULL）
<code>Channel* getOrCreateChannel(const std::string&)</code>	<code>Channel*</code>	B	Channel取得or作成
<code>void removeChannelIfEmpty(const std::string&)</code>	<code>void</code>	B	空Channel削除

4. C2層インターフェース（hanako担当）

詳細は [interface.md Section 8](#) 参照

4.1 Channel

関数	戻り値	呼び出し元	説明
<code>const std::string& name() const</code>	<code>const std::string&</code>	B, C1	channel名
<code>bool hasMember(Client*) const</code>	<code>bool</code>	B	参加済みか
<code>void addMember(Client*)</code>	<code>void</code>	B	member追加
<code>void removeMember(Client*)</code>	<code>void</code>	B	member削除
<code>void removeClient(Client*)</code>	<code>void</code>	B, C1	member/operator/invited一括削除
<code>std::vector<Client*> members() const</code>	<code>std::vector<Client*></code>	B	member一覧
<code>size_t memberCount() const</code>	<code>size_t</code>	B	member数
<code>bool isOperator(Client*) const</code>	<code>bool</code>	B	operator権限確認
<code>void addOperator(Client*)</code>	<code>void</code>	B	operator付与

関数	戻り値	呼び出し元	説明
<code>void removeOperator(Client*)</code>	<code>void</code>	B	operator剥奪
<code>void invite(Client*)</code>	<code>void</code>	B	招待リスト追加
<code>bool isInvited(Client*) const</code>	<code>bool</code>	B	招待済みか
<code>void removeInvite(Client*)</code>	<code>void</code>	B	招待解除
<code>void setTopic(const std::string&)</code>	<code>void</code>	B	topic設定
<code>const std::string& topic() const</code>	<code>const std::string&</code>	B	topic取得
<code>ChannelModes& modes()</code>	<code>ChannelModes&</code>	B	mode変更用
<code>const ChannelModes& modes() const</code>	<code>const ChannelModes&</code>	B	mode参照用
<code>bool isEmpty() const</code>	<code>bool</code>	B, C1	member 0人か

4.2 ChannelModes

関数	戻り値	呼び出し元	説明
<code>bool inviteOnly() const</code>	<code>bool</code>	B	+i状態
<code>bool topicRestricted() const</code>	<code>bool</code>	B	+t状態
<code>bool hasKey() const</code>	<code>bool</code>	B	+k状態
<code>const std::string& key() const</code>	<code>const std::string&</code>	B	パスワード
<code>int limit() const</code>	<code>int</code>	B	人数制限 (-1=無制限)
<code>void setInviteOnly(bool)</code>	<code>void</code>	B	+i/-i設定
<code>void setTopicRestricted(bool)</code>	<code>void</code>	B	+t/-t設定
<code>void setKey(const std::string&)</code>	<code>void</code>	B	+k設定
<code>void unsetKey()</code>	<code>void</code>	B	-k設定
<code>void setLimit(int)</code>	<code>void</code>	B	+l設定
<code>void unsetLimit()</code>	<code>void</code>	B	-l設定

5. 重要ルール（必ず守る）

5.1 nick変更は ServerState 経由

```
// ❌ NG
client.setNick("newNick");

// ✅ OK
state.updateNick(client, "newNick");
```

5.2 Client削除は ServerState 経由

```
// ServerState::removeClient(fd) が自動処理：
// - 全Channelからmember/operator/invited削除
// - fd辞書・nick辞書の更新
// - 空Channelの削除
```

5.3 operator権限は Channel が管理

```
// ❌ NG
client.setOperator(true);

// ✅ OK
channel.addOperator(&client);
```

5.4 Channel は Client を所有しない

- Channel は Client* を保持するだけ
- delete は ServerState の責務

6. 境界オブジェクト

6.1 CommandResult

B層からA層への戻り値。送信先fdと送信文字列を保持。

```
struct CommandResult {
    std::vector<OutgoingMessage> replies;
    bool shouldDisconnect;

    void addReply(int fd, const std::string& message);
};
```

6.2 OutgoingMessage

```
struct OutgoingMessage {
    int fd;
    std::string message;
};
```

7. 未確定事項（Open Questions）

項目	現状	決定タイミング
Poller分離	optional	実装時判断
ConnectionManager分離	optional	実装時判断
ClientRegistry分離	optional	実装時判断
ChannelService分離	optional	実装時判断

8. 関連ドキュメント

ドキュメント	内容
myIRCd/docs/interface.md	ベースドキュメント（詳細）
myIRCd/docs/design.md	全体設計・責務分割
onboarding_C1.md	taro向けオンボーディング
onboarding_C2.md	hanako向けオンボーディング
team_roles.md	チーム役割分担

変更履歴

日付	内容
2026-05-23	初版作成。myIRCd/docs/interface.md ベースに担当者割当追加